

Shi Cheng Huang

CSc 59866

Kurt Brown

Problem Background

The problem related to data science — predicting future values to make better decisions with using historical data in distributed computing — is the topic of resource usage prediction for autoscaling. Resource usage prediction for autoscaling is used to predict future CPU, memory, and I/O utilization so resources can be scaled proactively, rather than reacting after a failure occurs. If we give a workload too little, it slows down or crashes. If we give it too much, we waste resources and money. If we can predict the future demand for resources, then we can give each workload the right amount at the right time and will prevent failure due to resource has threshold from happening.

Cloud operators and platform engineers who run applications on distributed systems like Kubernetes (automates the scaling and management of containerized applications) experience this problem. They manage large numbers of workloads that each require CPU, memory, and I/O resources. They want to give each workload the right amount of resources — too little leads to slowdowns and crashes, while too much leads to wasted cost and capacity. It is currently done with a reactive method, in which the system watches resource usage and only adds resources when CPU or memory crosses a threshold. This is inefficient when many different workloads must be managed. The limitation is that by the time the system reacts, the problem has already occurred. This problem matters because it costs money and hurts users' experience when workloads receive the wrong amount of resources needed. The problem occurs under

conditions where resource demand changes so quickly that the system is unable to react, especially with unpredictable workloads.

Reactive autoscaling is structurally lagging, with a three-to-five-minute timing period spent waiting to scale resources after the threshold is breached. During this window, it leads to tail-latency degradation (the slowest performance) and service-level objective (SLO) violations. This claim is supported by Tran, Duy Anh as documented source. Conventional autoscaling, which is a reactive method, scales workloads based on the resource utilization of CPU and memory, which can lead to a failure to anticipate the burst demand of these workloads. This means that with the reactive method, the resources used for certain workloads cannot handle the burst demand for immediate changes in resources to achieve better performance. This claim is supported by Karthik Chakravarthy Cheekuri as documented source. With sudden load spikes, Horizontal Pod Autoscaler will trigger the threshold and wait for the new resource, which can cause cold starts, temporary under-provisioning, and frequent SLO violations. This claim is supported by Hyunsu Jeong and others as documented source.

Approach	How it addresses the problem	Strength	Limitations/Questions
Approach A (Kairos)	Use AI models and time-series for forecasting to adjust resource allocation.	Intelligent resource allocation, multiple safe scaling mode, and conflict avoidance.	AI dependency and forecast accuracy due to historical data.
Approach B (AROF)	Combine hybrid classification (clustering and	Multi-horizon forecasting to have	Model complexity with multi-horizon, as well with uncertainty

	tagging) and multi-horizon LSTM for forecasting with optimization.	many predictions over time.	especially for a long time into the future.
--	--	-----------------------------	---

The project currently appears closer to Option 1 because it is made to improve a distributed system tool by predicting the resource allocation for each workload for more efficient method than the reactive method, which costs more money and time with low performance especially the waiting time window for autoscaling. This is distributed scheduler that decides when and where to run the workload and how many resources they receive, which fits the resource usage prediction for autoscaling. The project currently appears to involve original design with element of the selection design. This is an original design because the task is to build a predictive autoscaling solution, which is existed in network (selection design) with like time-series model. Distributed computing might be important for this problem because the allocation of the resources for the workloads is not just only in a single machine setting, it has resources that are shared across many machines, services and workloads.

How many workloads are still using reactive autoscaling method for machine resources?

Answer: search the report that how many workloads are using reactive and others are proactive methods. Does a model work well in historical trace still work well in current workloads?

Answer: measure the accuracy with the resource allocation over time for the workloads. How do we know is the prediction of the number of the autoscaling can be trust? Answer: we can have both point prediction and prediction interval models to count the total number of SLO violation, and to compare the actual resource required and the predictable resource used.

The factors that can make the project difficult are difficulty evaluating the result, access to a realistic Kubernetes environment, and lack of required data. For difficulty evaluating the result, we don't have a controlled experimental environment to prove that predictive autoscaling has an improvement over reactive autoscaling. For access to a realistic Kubernetes environment, we are unable to run the experiment without access to a realistic Kubernetes environment. For lack of required data, since we can only get historical data to train the model, so it might not be appropriate to use in the live workload.

The discussion of selection design changed how I view this project because with using the right model that has exist as open-source in the network for my experiment on the resource usage prediction for autoscaling. The discussion of the principle of independent functionality (also called separation of concerns) can be used for separating prediction from decision making, and to test each component of the resource independently. The discussion of the risk management changed how I view this project because with current setting and environment, it will be difficult to implement certain parts of the experiments. With how to control the risk that occurs during the experiment.

Reference:

Tran, Duy Anh. "Predictive autoscaling for kubernetes using machine learning workload forecasting", Trepo, <https://trepo.tuni.fi/handle/10024/238698#1>

Karthik Chakravarthy Cheekuri. "SLO-First Autoscaling for Multi-Tenant Microservices: A Control-Theoretic Approach to P95/P99 Latency", Journal of Information Systems Engineering and Management, <https://www.jisemjournal.com/index.php/journal/article/view/13473/6334>

Jeong et al. "SLO-Aware and Cost-Efficient Predictive Autoscaling for Kubernetes Microservices", Jeju National University, <https://www.sc-asia.org/2026/data/poster/post240.pdf#1#1>